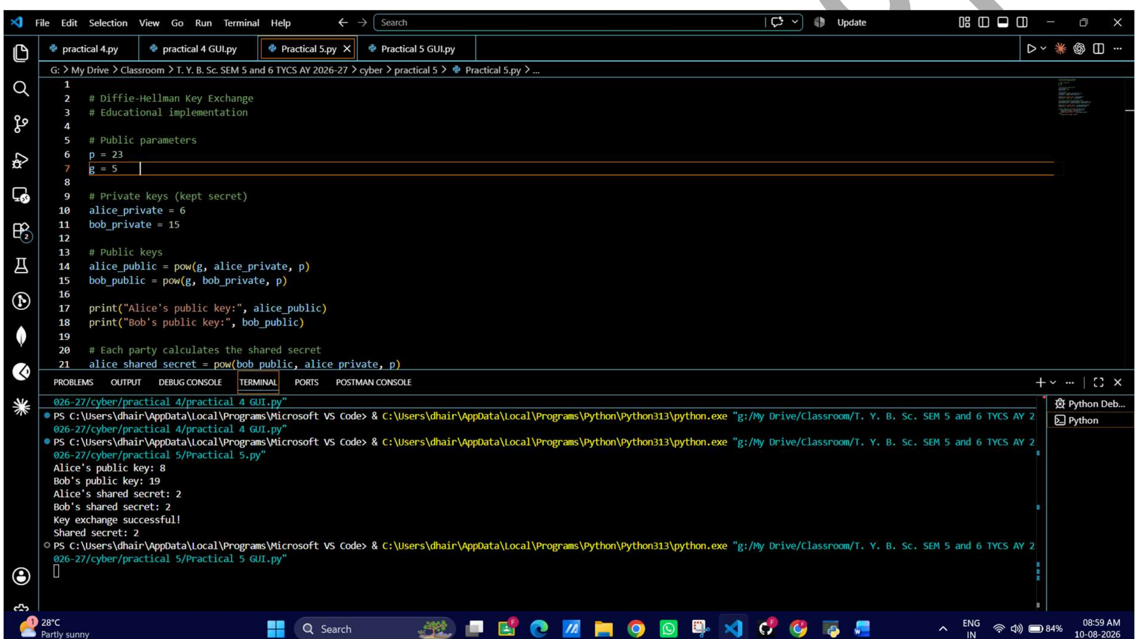
Aim: Implement the Diffie-Hellman key exchange algorithm to securely exchange keys between two entities over an insecure network.

CLI:

```
# Diffie-Hellman Key Exchange

# Educational implementation


# Public parameters

p = 23

g = 5


# Private keys (kept secret)

alice_private = 6

bob_private = 15


# Public keys

alice_public = pow(g, alice_private, p)

bob_public = pow(g, bob_private, p)


print("Alice's public key:", alice_public)

print("Bob's public key:", bob_public)


# Each party calculates the shared secret

alice_shared_secret = pow(bob_public, alice_private, p)

bob_shared_secret = pow(alice_public, bob_private, p)


print("Alice's shared secret:", alice_shared_secret)

print("Bob's shared secret:", bob_shared_secret)
```

```
# Verify that both calculated the same secret

if alice_shared_secret == bob_shared_secret:

    print("Key exchange successful!")

    print("Shared secret:", alice_shared_secret)

else:

    print("Key exchange failed!")
```

Output:



GUI:

```
import tkinter as tk
from tkinter import messagebox


def perform_diffie_hellman():
    try:
        # Get values from GUI
        p = int(p_entry.get())
        g = int(g_entry.get())
        alice_private = int(alice_entry.get())
        bob_private = int(bob_entry.get())

        # Basic validation
        if p <= 1:
            raise ValueError("p must be greater than 1.")

        if g <= 0:
            raise ValueError("g must be positive.")

        if alice_private <= 0 or bob_private <= 0:
            raise ValueError("Private keys must be positive.")

        # Calculate public keys
        alice_public = pow(g, alice_private, p)
        bob_public = pow(g, bob_private, p)

        # Calculate shared secret
        alice_shared = pow(bob_public, alice_private, p)
```

```
        bob_shared = pow(alice_public, bob_private, p)

        # Display results
        alice_public_result.config(text=str(alice_public))
        bob_public_result.config(text=str(bob_public))
        alice_shared_result.config(text=str(alice_shared))
        bob_shared_result.config(text=str(bob_shared))

        if alice_shared == bob_shared:
            status_label.config(
                text="✓ Key Exchange Successful!",
                fg="green"
            )
        else:
            status_label.config(
                text="✗ Key Exchange Failed!",
                fg="red"
            )

    except ValueError as e:
        messagebox.showerror("Invalid Input", str(e))

def clear_fields():
    p_entry.delete(0, tk.END)
    g_entry.delete(0, tk.END)
    alice_entry.delete(0, tk.END)
    bob_entry.delete(0, tk.END)
```

```
    alice_public_result.config(text="-")
    bob_public_result.config(text="-")
    alice_shared_result.config(text="-")
    bob_shared_result.config(text="-")

    status_label.config(text="")

# ---------------- GUI ----------------

root = tk.Tk()
root.title("Diffie-Hellman Key Exchange")
root.geometry("600x600")
root.resizable(False, False)
root.configure(bg="#f2f2f2")

# Title
title = tk.Label(
    root,
    text="Diffie-Hellman Key Exchange",
    font=("Arial", 22, "bold"),
    bg="#f2f2f2",
    fg="#1f3c88"
)
title.pack(pady=20)

subtitle = tk.Label(
    root,
    text="Secure key exchange demonstration",
```

```
    font=("Arial", 11),
    bg="#f2f2f2",
    fg="#555555"
)
subtitle.pack()

# Input frame
input_frame = tk.Frame(root, bg="white", padx=20, pady=20)
input_frame.pack(pady=20)

# Public parameters
tk.Label(
    input_frame,
    text="Public Parameters",
    font=("Arial", 14, "bold"),
    bg="white"
).grid(row=0, column=0, columnspan=2, pady=10)

tk.Label(
    input_frame,
    text="Prime number (p):",
    font=("Arial", 11),
    bg="white"
).grid(row=1, column=0, sticky="w", pady=7)

p_entry = tk.Entry(input_frame, width=25, font=("Arial", 11))
p_entry.grid(row=1, column=1, pady=7)
```

```
tk.Label(
    input_frame,
    text="Generator (g):",
    font=("Arial", 11),
    bg="white"
).grid(row=2, column=0, sticky="w", pady=7)

g_entry = tk.Entry(input_frame, width=25, font=("Arial", 11))
g_entry.grid(row=2, column=1, pady=7)

# Private keys
tk.Label(
    input_frame,
    text="Private Keys",
    font=("Arial", 14, "bold"),
    bg="white"
).grid(row=3, column=0, columnspan=2, pady=(15, 10))

tk.Label(
    input_frame,
    text="Alice's private key:",
    font=("Arial", 11),
    bg="white"
).grid(row=4, column=0, sticky="w", pady=7)

alice_entry = tk.Entry(input_frame, width=25, font=("Arial", 11))
alice_entry.grid(row=4, column=1, pady=7)
```

```
tk.Label(
    input_frame,
    text="Bob's private key:",
    font=("Arial", 11),
    bg="white"
).grid(row=5, column=0, sticky="w", pady=7)

bob_entry = tk.Entry(input_frame, width=25, font=("Arial", 11))
bob_entry.grid(row=5, column=1, pady=7)

# Buttons
button_frame = tk.Frame(root, bg="#f2f2f2")
button_frame.pack(pady=5)

calculate_button = tk.Button(
    button_frame,
    text="Calculate Shared Key",
    command=perform_diffie_hellman,
    bg="#1f6feb",
    fg="white",
    font=("Arial", 11, "bold"),
    padx=15,
    pady=8
)
calculate_button.grid(row=0, column=0, padx=10)

clear_button = tk.Button(
    button_frame,
```

```
    text="Clear",
    command=clear_fields,
    bg="#555555",
    fg="white",
    font=("Arial", 11, "bold"),
    padx=25,
    pady=8
)
clear_button.grid(row=0, column=1, padx=10)

# Results
result_frame = tk.Frame(root, bg="white", padx=25, pady=15)
result_frame.pack(pady=15)

tk.Label(
    result_frame,
    text="Results",
    font=("Arial", 14, "bold"),
    bg="white"
).grid(row=0, column=0, columnspan=2, pady=5)

tk.Label(
    result_frame,
    text="Alice's Public Key:",
    font=("Arial", 11),
    bg="white"
).grid(row=1, column=0, sticky="w", pady=5)
```

```
alice_public_result = tk.Label(
    result_frame,
    text="-",
    font=("Arial", 11, "bold"),
    bg="white",
    fg="#1f3c88"
)
alice_public_result.grid(row=1, column=1, sticky="w", padx=20)

tk.Label(
    result_frame,
    text="Bob's Public Key:",
    font=("Arial", 11),
    bg="white"
).grid(row=2, column=0, sticky="w", pady=5)

bob_public_result = tk.Label(
    result_frame,
    text="-",
    font=("Arial", 11, "bold"),
    bg="white",
    fg="#1f3c88"
)
bob_public_result.grid(row=2, column=1, sticky="w", padx=20)

tk.Label(
    result_frame,
    text="Alice's Shared Secret:",
```

```
    font=("Arial", 11),
    bg="white"
).grid(row=3, column=0, sticky="w", pady=5)

alice_shared_result = tk.Label(
    result_frame,
    text="-",
    font=("Arial", 11, "bold"),
    bg="white",
    fg="#8a2be2"
)
alice_shared_result.grid(row=3, column=1, sticky="w", padx=20)

tk.Label(
    result_frame,
    text="Bob's Shared Secret:",
    font=("Arial", 11),
    bg="white"
).grid(row=4, column=0, sticky="w", pady=5)

bob_shared_result = tk.Label(
    result_frame,
    text="-",
    font=("Arial", 11, "bold"),
    bg="white",
    fg="#8a2be2"
)
bob_shared_result.grid(row=4, column=1, sticky="w", padx=20)
```

```
# Status
status_label = tk.Label(
    root,
    text="",
    font=("Arial", 13, "bold"),
    bg="#f2f2f2"
)
status_label.pack(pady=10)


root.mainloop()
```


Output: